UNIVERSIDAD NACIONAL AUTÓNOMA DE MEXICO

Introducción a ciencias de la computación

Nicolás Bustos

Número de cuenta 427340976

Práctica de laboratorio: Decisiones y repeticiones

Parte 1: Calculando pi a dardazos

1.1 Un numero fijo de dardos

Num. de experimento	Número de dardos	Estimación de pi
1	100	3.280000
2	100	2.920000
3	100	3.600000
4	1,000	3.196000
5	1,000	3.212000
6	1,000	3.140000
7	10,000	3.151600
8	10,000	3.165200
9	10,000	3.159200
10	100,000	3.138720
11	100,000	3.134920
12	100,000	3.149640
13	1,000,000	3.142540
14	1,000,000	3.143008
15	1,000,000	3.146108

Observamos que el resultado varía de ejecuciónón en ejecución porque depende de un número generado aleatoriamente entre 0 y 1. Mientras más sean los dardos lanzados, la aproximación es más precisa porque tenemos más valores de referencia que en una curva de distribución eliminan la significancia de cada valor que se aleja mucho de la desviación estándar.



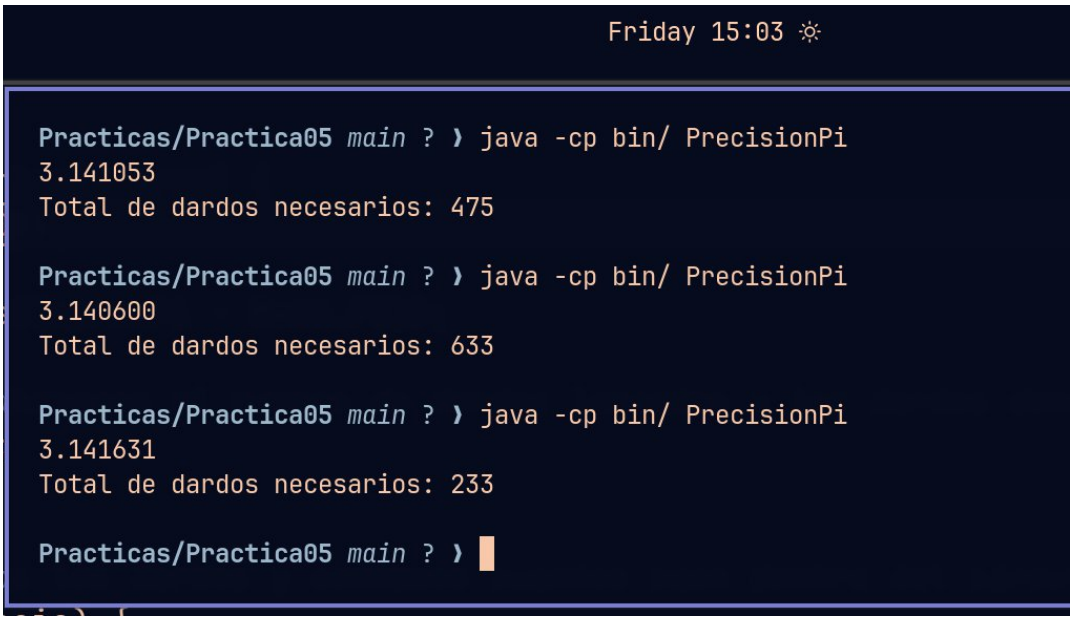
1.2 Repetir hasta llegar a la precisión deseada

La estrategia para escribir este programa es usar el ciclo while requerido mientras el error en mi estimación de pi sea más grande que la tolerancia que le permito. Cuando esto sucede, el programa va aproximando pi con más precisión cada vez, hasta lograr el valor deseado.

Tolerancia	Ejecución	Dardos Necesarios	Estimacion de Pi
0.1	1	5	3.200000
0.1	2	9	3.111111
0.1	3	5	3.200000
	Promedio de dardos:	6.33...	
0.01	4	14	3.142857
0.01	5	28	3.142857
0.01	6	42	3.142857
	Promedio de dardos:	28	
0.001	7	475	3.141053
0.001	8	633	3.140600
0.001	9	233	3.141631
	Promedio de dardos:	447	

En ninguna de las ejecuciones hubo una variación muy grande del número de veces necesarias de lanzar el dado para cumplir con la tolerancia dada, aunque al ser números generados aleatoriamente pudo haber una diferencia muy grande por casualidad que no implicara que el programa tuviera fallas. En este caso, el promedio de dardos lanzados ejemplifica bien lo que sucedió, ya que se encuentra más o menos a la mitad entre el caso con menos dardos y el caso con más dardos.

Al comparar los resultados de este programa con el programa anterior en el que el cálculo se detenía al llegar a un número dado de dardos, podemos ver que hay diferencias importantes. Para el programa basado en la tolerancia de la aproximación, necesitamos muchos menos dardos para obtener un resultado más preciso. Esto sucede ya que al promediar muchos datos, algunos de esos datos nos pueden alejar más del resultado esperado, y en el caso del programa basado en tolerancia, deja de añadir más datos que pueden confundirnos, al llegar al número requerido.

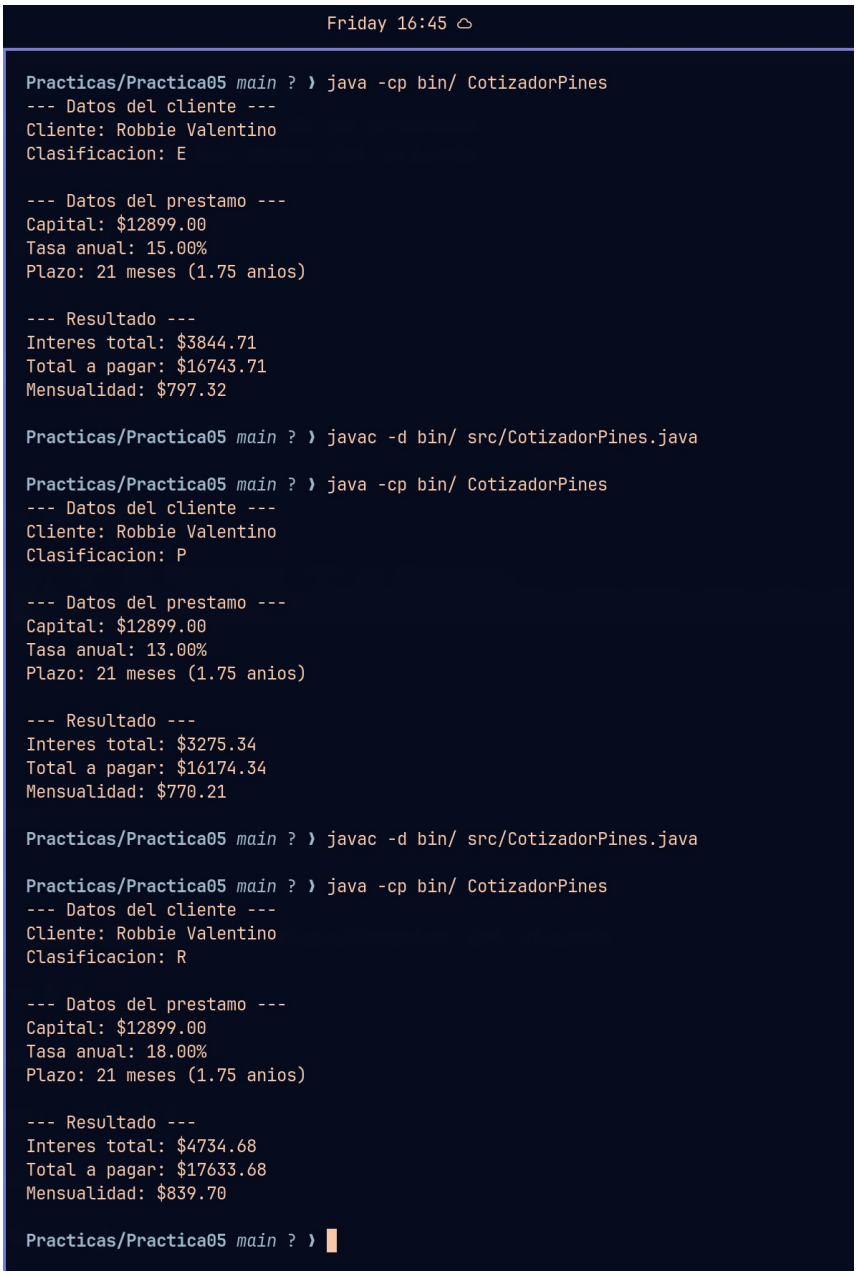


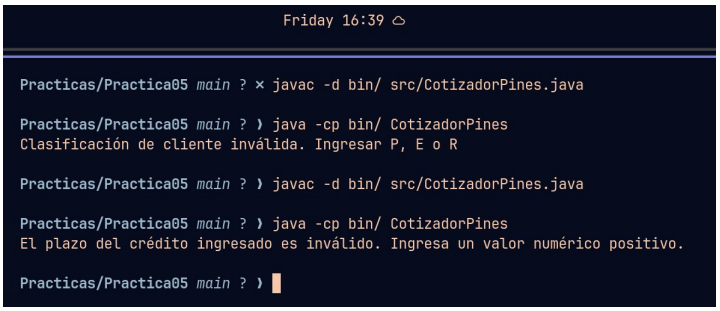
Parte 2: El regreso del señor Pines

2.1 Verificar los datos y el tipo de cliente

Los datos a revisar elegidos fueron Clasificación del cliente, Plazo del crédito y Capital, porque son los principales datos que modificamos al registrar una nueva transacción, sin contemplar al nombre porque es una cadena de texto.

Para revisar que estén correctos; en el caso de la clasificación del cliente uso un if-else anidado para validar si es P, E o R, y actualizar la tasa de interés para los casos P y R, y en cualquier otro caso asigno un error. En el caso del plazo del crédito y el capital, con un if valido si el valor ingresado es mayor a cero, y para cualquier otro caso (else) asigno error.



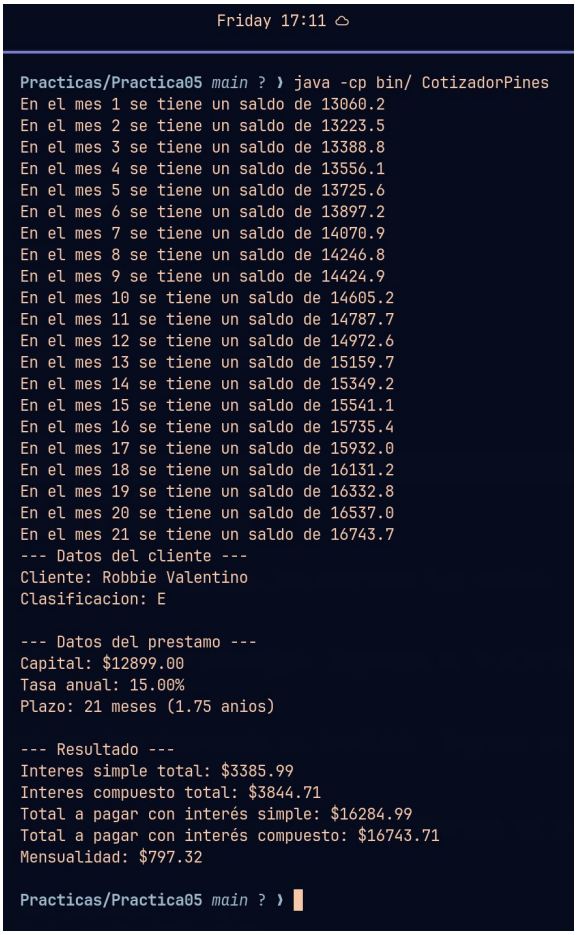


En la ejecución vemos que cuando es introducido un dato inválido, recibimos únicamente la explicación del error y la solicitud para que se haga la corrección. Cuando los resultados son correctos, el programa hizo el cálculo solicitado e imprimió los resultados.

2.2 Interés compuesto

Para diseñar el código del interés compuesto y que imprima cada iteración, lo primero que hice fue añadir un if en el que si había un error en los datos, no entrara al bucle. Una vez validamos que los datos ingresados sean correctos, inicia el ciclo, en el que calculamos el interés por cada mes transcurrido y lo sumamos al cálculo del siguiente en cada iteración.

Luego uso la fórmula del interés compuesto para evitar el uso del bucle for.



Observamos que ambas salidas coinciden (16,743.7 en el ciclo for y 16,743.71 usando la fórmula), con un error de 0.01 por el formato en el que imprimimos el bucle for (hemos especificado imprimir solo un decimal).

CONCLUSIONES

La parte dos de la práctica, sobre el cotizador, fue muy interesante, porque tuvimos que pensar en una solución por nosotros mismos para lograr que el programa solo imprimiera los errores sin el resto de las operaciones y resultados. Obligarme a comentar el código y seguir las buenas prácticas en esta actividad fue desafiante, por no estar acostumbrado a pensar en qué cosas están claras con el mismo código y cuáles debo explicar textualmente, y qué tanto me debo extender en cada comentario.

Sobre la práctica 1, fue muy satisfactorio poder implementar un método matemático formal desde un programa escrito por nosotros mismos.